

Shaurya Doger

Experiment 2

3/26/2023

Professor Jae rock Kwon

Experiment 2 Report

This report below will describe my 'Experiment 2' and all the steps I took to get my microcontroller functionality with 'LED' and 'pins':

- 1) "PortD_Pin0_Off" function is contained in "PortD.asm" file, which the function will be described below:

```
-----PortD_Pin0_Off-----
; Turn the Port D Pin0 off
; Input: R0, which takes in the current value at "R1 memory address"
; Output: none
; Modifies: R0, R1
PortD_Pin0_Off: .asmfunc
    LDR R1, GPIO_PORTD_DATA_R      ; R1 = &GPIO_PORTD_DATA_R
    LDR R0, [R1]                   ; R0 = [R1]
    AND R0, R0, #0xFE               ; R0 = R0|0x00 friendly set pin_0 to 0, with other bits set to 1 (AND) to not
    STR R0, [R1]                   ; [R1] = R0

    BX LR
.endasmfunc
```

With the code shown above, "register-1" takes in the memory address of "Port-D Data Register", then "register-0" takes in the "bit-value" in that particular memory address, followed by 'AND' operation that sets value for "pin-0" to turn off while other bits not affected, and then finally that 'R0' value gets loaded at the memory address of "Port-D Data Register".

- 2) "green_led_toggle" and "timed_green_led_toggle" functions are contained in "main.asm" file which are shown below:

```
-----timed_green_led_toggle-----
; Toggle the green LED based on the amount of delay time provided as an input to the function, and subsequently it
; It calls the 'green_led_toggle' function from below!!!
; Input: R1, which takes in the current value at "GPIO_PORTF_DATA_R memory address" with 'green_led_toggle'
; Output: It writes "toggled R1" value to the "R0" memory address at 'green_led_toggle'!!!
; Modifies: R0, R1
timed_green_led_toggle: .asmfunc
    BL green_led_toggle
    ;LDR R10, ONESEC
    BL delay
    B MainLoop
    ;BX LR
.endasmfunc
```

The above "timed_green_led_toggle" function calls "green_led_toggle" function before calling "delay" function from the "LED.asm" file. Then, this "timed_green_led_toggle" returns to the 'MainLoop' where "green_led" flash rate is based upon "SW1" button status.

```

;-----green_led_toggle-----
; Toggle the green LED where turns on if off, or off when currently on
; Input: R1, which takes in the current value at "GPIO_PORTF_DATA_R memory address"
; Output: It writes "toggled R1" value to the "R0" memory address!!!
; Modifies: R0, R1
green_led_toggle: .asmfunc
    LDR R0, GPIO_PORTF_DATA_R
    LDR R1, [R0]
    EOR R1, R1, #GREEN
    STR R1, [R0]

    BX LR
.endasmfunc

```

The above function loads “PortF_Data_Register” address into “R0”, and then the “addresses-current-value” with “R1”. The “EOR” instruction line toggles the “green LED bit” (Pin 3) where “GREEN” value is set to “1 if 0” and vice versa. Lastly, the updated “R1” value gets put into the address, then the ‘LED’ toggles.

```

1 ;-----delay-----
2
3 ; The delay count is passed to this function through R10
4 delay: .asmfunc
5     ;LDR R10, POINTONESEC
6 delay_loop
7     SUBS R10, R10, #1                ; R10 = R10 - 1 (count = count - 1)
8     BNE delay_loop                ; if count (R10) != 0, skip to 'delay'
9     BX LR                        ; return
10    .endasmfunc

```

The above “delay” function takes in “R10” value that was loaded during the “MainLoop”, then continuously subtracts 1 “until 0”, then the function exits. Each “flash rate” has different value that would get loaded to “R10”. This function is contained in the “LED.asm” file.

- 3) The mechanisms utilized for passing input parameters are done through the following with “MainLoop” that does the following as described below:

```

MainLoop
;Add your code using SW1 to control the GREEN LED flashing rate
;Your code should read SW1
;If SW1 is open(not pressed) R10 should be set to the count for a 1 sec delay
;Else R10 should set to the count for a 0.1 sec delay
;Use the If Then Else structure discussed in lecture
;The starter code only flashes the LED at one rate

;BL green_led_toggle
;BL green_led_toggle

    LDR R1, GPIO_PORTF_DATA_R ; R1 = &GPIO_PORTF_DATA_R
    LDR R0, [R1] ; R0 = [R1]!!! This line reads in value from SW1. If pressed, then 0x01. If not pressed, then 0x11.
    ANDS R0, R0, #0x10 ; R0 = R0&0x10!!! If SW1 pressed, R0 will contain a value of zero with this AND operation!!!
    BNE SW1_open ;This basically says if R0 contains a value of zero, it will not go to 'SW1_open'. Otherwise it will.
    LDR R10, POINTONESEC ; 0.1 sec delay
    B next

SW1_open
    LDR R10, ONESEC ; 1 sec delay
next
    BL timed_green_led_toggle

    B MainLoop

```

The above image shows the mechanism behind where “SW1” input decides if the “green_led” flashes at “1 sec” or “0.1 sec”. In the overall code, main registers that get utilized include “R1”, “R0”, “R10”, and “Program Status Register’s Zero Status Flag”. The “General-Purpose Registers” described are used for loading memory addresses, their specific values, and ‘timing-values’. “ANDS” instruction not only does “&-operation” but also updates the “Z-flag” within “Program Status Register” so that it knows whether value is a zero ‘or’ not.

“MainLoop” starts through loading “PortF_Data_R” memory address into “R1”, and then load that addresses value into “R0”. The “ANDS” operation with “0x10” determines value into “R0” and depending on if “SW1” is pressed or not, the “R0” value may/may not be a zero. This determines whether it will branch to “SW1_open” and if it does, then “R10” gets loaded with “ONESEC” before calling the “timed_green_led_toggle” function. With “Branch Not Equal”, when “R0” has ‘0’ value, it means button is pressed so “0.1 sec” value gets uploaded into “R10” for ‘delay’ function to make ‘green_led’ to flash at a faster rate.

- 4) The following YouTube links showcase demonstration on how the microcontroller ‘PINS’ and ‘LED’ function:
 - a. This video link demonstrates the ‘Q1’ concept:
<https://www.youtube.com/watch?v=QfTBuckDEIU>
 - b. This video demonstrates ‘Q2/Q3’ concept:
https://www.youtube.com/watch?v=UQVI_U6oUoU
- 5) Overall, the completion of this experiment took quite a bit of effort to complete and get correct functionality with my microcontroller. The most challenging aspects of this were mastering “TM4C123” architecture and proper techniques for debugging across specific lines of code. Other difficulties with this project include “development of efficiency with looping structures”, comprehension behind “button-press” concepts, and “time-based-toggle-flashes” with “green-led”. However, through research on specific “port-pin” concepts and particular “Assembly Language Commands”, I was able to overcome them as I could debug specific lines and even set breakpoints to further investigate any issues that arose in my functionality. I have learned a lot from this project in terms of how the microcontroller works, commands, interpretation of ‘binary/hex values’ with ‘inputs’ and ‘outputs’, and that I can use ‘Memory/Register Browser’ to investigate at a deeper level to what is occurring with specific parts of microcontroller.